Intro to SQL Programming CheatSheet

{ik} INTERVIEW KICKSTART

Table Of Content

- Introduction to SQL
- SQL Programming Basics
- Combining Multiple Tables
- Combining Multiple Queries
- Advanced Row Combinations
- Order of Execution in SQL
- SQL Transactions
- SQL Security Best Practices
- Best Practices
- Additional Resources

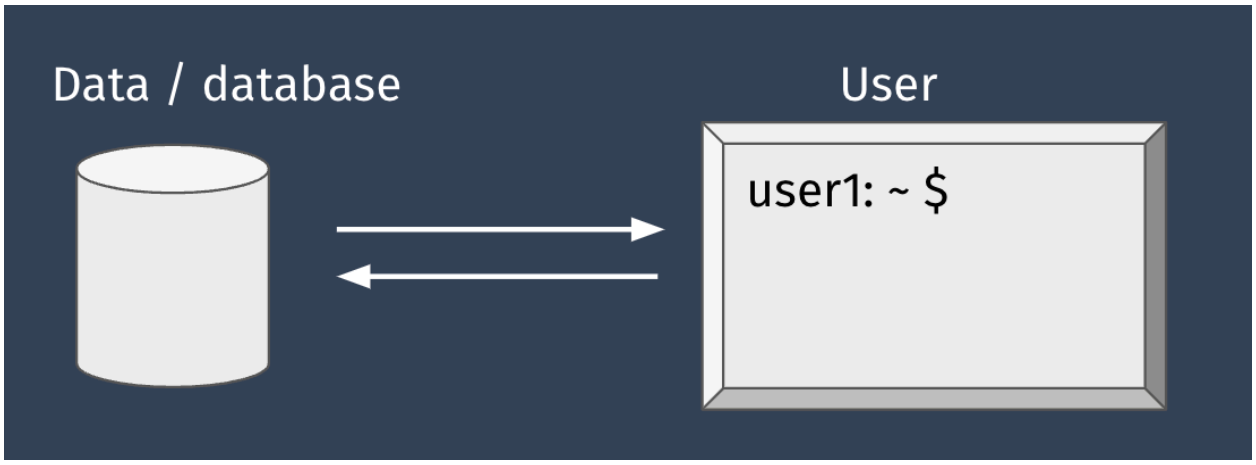
Prerequisites

1. **Install PostgreSQL**
   - Ensure that PostgreSQL is installed on your system.
   - Follow the detailed **PostgreSQL Software Installation Guide** to set up PostgreSQL on your machine.

2. **Setup the IMDB Database**
   - You will need to set up the IMDB database before the class.
   - Step 1: Download the database file by clicking on the following link: **IMDB_DB.sql**.
   - Step 2: Follow the steps in the **PostgreSQL IMDB Database Setup Guide** to properly set up the database in PostgreSQL.

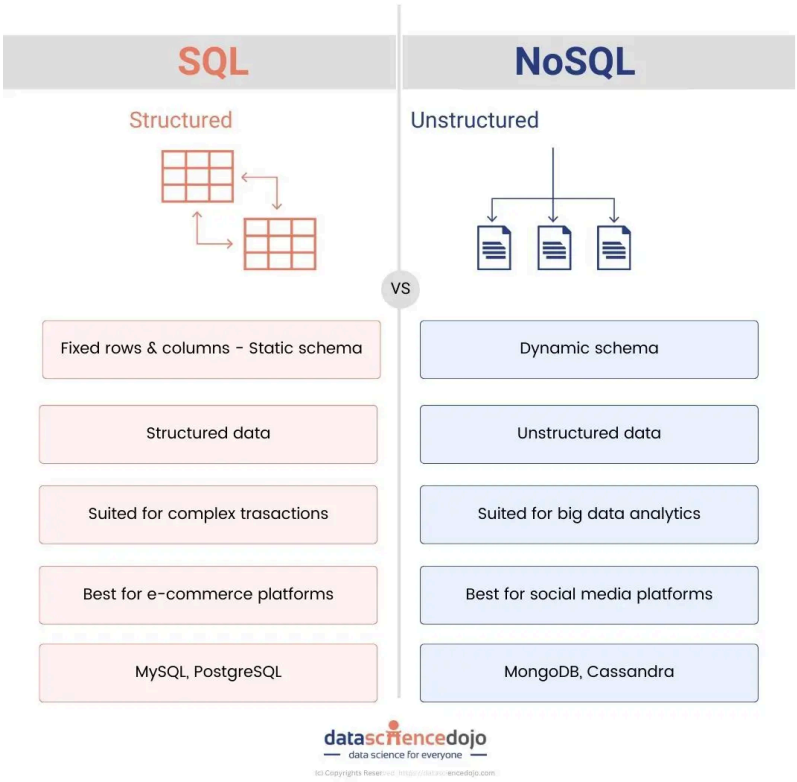
Introduction to SQL

SQL (Structured Query Language) is a standardized language used to manage and manipulate relational databases. It allows for querying, updating, and managing data efficiently, making it a cornerstone of data management and analytics.



SQL vs NoSQL

- **SQL:** Structured data with a fixed schema, strong ACID compliance, and support for complex queries and transactions.
- **NoSQL:** Flexible schema, horizontal scalability, often used in large-scale distributed data environments where performance and flexibility are prioritized over consistency.



SQL Database Systems

- **Popular RDBMS:** PostgreSQL, MySQL, Oracle, Microsoft SQL Server, SQLite.
- **SQL Syntax Variations:** Each RDBMS might have slight differences in SQL syntax (e.g., `LIMIT` in MySQL vs. `FETCH FIRST` in Oracle).

Common SQL Commands

- **Data Definition Language (DDL):** Commands like `CREATE`, `ALTER`, `DROP`, and `TRUNCATE` are used to define the structure of databases and tables.
- **Data Manipulation Language (DML):** Commands like `INSERT`, `UPDATE`, `DELETE` are used to manipulate data within the tables.
- **Data Query Language (DQL):** The `SELECT` statement is used to query the database and retrieve data.
- **Data Control Language (DCL):** Commands like `GRANT`, `REVOKE` are used to control access to data in the database.

Data Types in SQL

- **Numeric Types:** INT, FLOAT, DECIMAL - used for storing numbers.
- **Date and Time Types:** DATE, TIME, TIMESTAMP - used for storing date and time information.
- **String Types:** CHAR, VARCHAR, TEXT - used for storing text data.
- **Boolean Types:** BOOLEAN - used for storing true/false values.
- **Binary Types:** BLOB, BYTEA - used for storing binary data like images or files.

SQL Programming Basics

SELECT Queries

- The `SELECT` statement is used to fetch data from a database.
- Example: `SELECT name, age FROM employees WHERE department = 'HR';`
- The `DISTINCT` keyword can be used to return only unique values.
- **Example:**

```
SELECT DISTINCT department FROM
employees;
```

Filtering with WHERE, AND, OR

- The `WHERE` clause is used to filter records based on specific conditions.
- `AND` and `OR` operators are used to combine multiple conditions.
- **Example:**

```
SELECT * FROM orders WHERE status =
'Shipped' AND order_date > '2023-01-01';
```

Aggregating Data

- Aggregate functions like `SUM`, `COUNT`, `AVG`, `MIN`, `MAX` are used to perform calculations on a set of values.
- **Example:**

```
SELECT COUNT(*), department FROM
employees GROUP BY department HAVING
COUNT(*) > 10;
```

Sorting and Limiting Results

- `ORDER BY` is used to sort the result set in ascending or descending order.
- `LIMIT` restricts the number of rows returned in the result set.
- Example:

```
SELECT * FROM products ORDER BY price
DESC LIMIT 5;
```

SQL Functions

- **Scalar Functions:** Operate on a single value and return a single value. Examples include `UPPER()`, `LOWER()`, `ROUND()`, `LEN()`.
- **Aggregate Functions:** Perform a calculation on a set of values and return a single value. Examples include `SUM()`, `AVG()`, `COUNT()`.
- **Date Functions:** `NOW()`, `DATEADD()`, `DATEDIFF()`, `CURDATE()`.
- **String Functions:** `CONCAT()`, `SUBSTRING()`, `TRIM()`, `REPLACE()`.

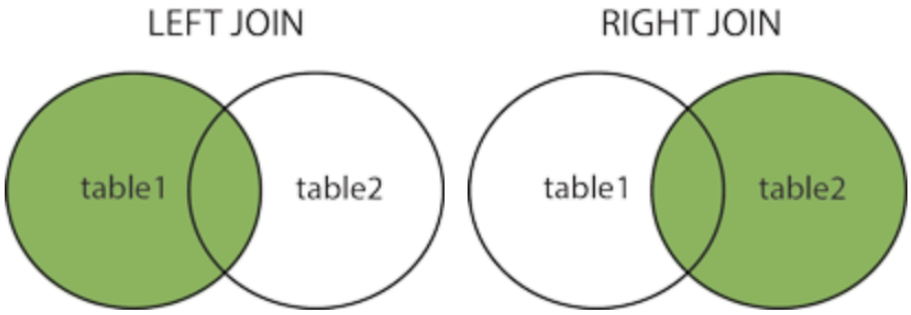
Combining Multiple Tables

SET Operations

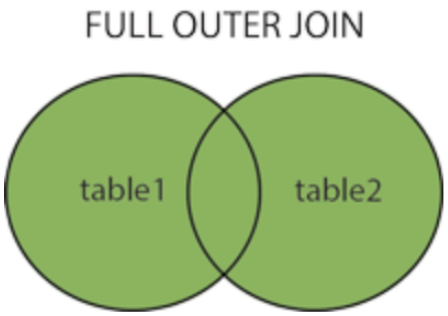
- **UNION:** Combines the results of two queries into a single result set and removes duplicates.
- **UNION ALL:** Similar to `UNION`, but includes duplicates.
- **INTERSECT:** Returns only the rows that are common to both queries.
- **EXCEPT (MINUS):** Returns the rows from the first query that are not present in the second query.
- Example: `SELECT name FROM employees WHERE department = 'Sales' UNION SELECT name FROM employees WHERE department = 'Marketing';`

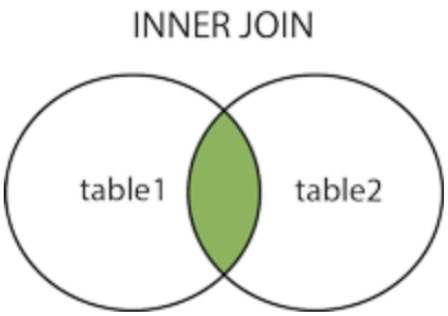
JOIN Operations

- **INNER JOIN:** Returns records that have matching values in both tables.
- **LEFT JOIN (or LEFT OUTER JOIN):** Returns all records from the left table, and the matched records from the right table.
- **RIGHT JOIN (or RIGHT OUTER JOIN):** Returns all records from the right table, and the matched records from the left table.
- **FULL JOIN (or FULL OUTER JOIN):** Returns all records when there is a match in either the left or right table.



{ik} INTERVIEW KICKSTART






```
SELECT employees.name,
departments.department_name
FROM employees
INNER JOIN departments ON
employees.department_id =
departments.id;
```

Common Pitfalls with JOINs

- **Cartesian Product:** Forgetting to specify the join condition, leading to a Cartesian product.
- **Ambiguous Column References:** When columns from multiple tables have the same name, always use table aliases.

Combining Multiple Queries

Nested Queries (Subqueries)

- Subqueries are used within the WHERE clause or HAVING clause to perform a query within another query.
- **Example:**

```
SELECT employee_id FROM employees WHERE
salary > (SELECT AVG(salary) FROM
employees);
```

WITH Clauses

- `WITH` clause allows you to define temporary result sets that can be referenced within a SELECT, INSERT, UPDATE, or DELETE statement.

  Example:

```
WITH Sales_CTE AS (
    SELECT product_id, SUM(sales_amount)
AS total_sales
    FROM sales
    GROUP BY product_id
)
SELECT * FROM Sales_CTE WHERE
total_sales > 1000;
```

Creating Views

- A VIEW is a virtual table based on the result set of a SQL query.

```
CREATE VIEW HighValueOrders AS
SELECT order_id, customer_id,
order_total
FROM orders
WHERE order_total > 1000;
```

Temporary Tables and CTEs

- Temporary tables are used to store intermediate results temporarily within a session.
- CTEs (Common Table Expressions) allow for creating a temporary result set that can be referenced within the main query.

Advanced Row Combinations

Window Functions

- Window functions perform a calculation across a set of table rows that are somehow related to the current row.
- Examples include `RANK()`, `DENSE_RANK()`, `ROW_NUMBER()`, `LAG()`, `LEAD()`.
- **Example:**

```
SELECT product_id, sales, RANK() OVER
(ORDER BY sales DESC) FROM products;
```

Analytical Functions

- These functions provide insights into data over partitions or windows, such as calculating running totals, moving averages, or ranking.

```
SELECT product_id, sales,
       SUM(sales) OVER (PARTITION BY
category_id ORDER BY sales_date) AS
running_total
FROM sales;
```

Common Use Cases

- **Top N Products per Category:** `RANK() OVER (PARTITION BY category ORDER BY sales DESC)`
- **Rolling Sum:** `SUM(sales) OVER (ORDER BY sales_date)`

Recursive Queries

- Recursive queries are used to query hierarchical data, such as organizational charts or file systems.

  Example:

```sql
WITH RECURSIVE OrgChart AS (
    SELECT employee_id, manager_id,
employee_name
    FROM employees
    WHERE manager_id IS NULL
    UNION ALL
    SELECT e.employee_id, e.manager_id,
e.employee_name
    FROM employees e
    INNER JOIN OrgChart o ON
o.employee_id = e.manager_id
)
SELECT * FROM OrgChart;
```

Order of Execution in SQL

1. Execution Order

- FROM and JOINs
- WHERE
- GROUP BY
- HAVING
- SELECT
- DISTINCT
- ORDER BY
- LIMIT
- SET operations (with their SELECT statements)

Examples

- **Example 1:**

```
SELECT product_id, COUNT(*) FROM sales
WHERE sales_date > '2023-01-01' GROUP BY
product_id HAVING COUNT(*) > 5 ORDER BY
COUNT(*) DESC;
```

- Execution: Access the table, filter by date, group by product_id, filter groups, and sort the results.

Common Performance Optimization Techniques

- **Indexes:** Create indexes on columns that are frequently used in WHERE, JOIN, and ORDER BY clauses.
- **Query Plan Analysis:** Use the `EXPLAIN` command to analyze how a query is executed and identify bottlenecks.
- **Avoiding Full Table Scans:** Ensure that queries are optimized to avoid scanning the entire table when unnecessary.

SQL Transactions

ACID Properties

- **Atomicity:** Ensures that a transaction is either fully completed or fully rolled back.
- **Consistency:** Ensures that a transaction brings the database from one valid state to another.
- **Isolation:** Ensures that concurrent transactions do not interfere with each other.
- **Durability:** Ensures that the results of a committed transaction are permanently stored in the database.

Transaction Control Commands

- **COMMIT:** Permanently saves all the changes performed by the transaction.
- **ROLLBACK:** Undoes the changes made by the transaction.
- **SAVEPOINT:** Sets a savepoint within a transaction to which you can roll back.

Handling Deadlocks and Concurrency Issues

- **Deadlocks:** Occur when two or more transactions are waiting for each other to release locks.
- **Concurrency Control:** Techniques include locking, isolation levels, and optimistic concurrency control to manage concurrent transactions.

SQL Security Best Practices

Preventing SQL Injection

- Always use parameterized queries or prepared statements to prevent SQL injection attacks.
- Avoid dynamic SQL where possible, or ensure proper escaping of user input.

Role-Based Access Control (RBAC)

- Assign roles to users and grant permissions to roles instead of individual users.

Example:

```
GRANT SELECT, INSERT ON database_name TO
role_name;
```

Data Encryption Techniques

- **Encryption at Rest:** Encrypt data stored on disk to protect it from unauthorized access.
- **Encryption in Transit:** Use SSL/TLS to encrypt data as it moves between the client and the server.

Auditing and Logging SQL Queries

- Enable logging of SQL queries to monitor for suspicious activity.
- Use database auditing features to track changes to sensitive data.

Best Practices

1. Tips for Writing Efficient SQL Queries

- **Select Only What You Need:** Avoid `SELECT *`; specify the required columns.
- **Index Usage:** Use indexes to speed up query performance.
- **Avoid Subqueries in SELECT:** Use joins or WITH clauses instead.

2. Common Pitfalls

- **Ignoring Data Types:** Ensure correct data types are used to avoid implicit conversions.
- **Overuse of Cursors:** Avoid using cursors unless necessary, as they can slow down performance.

3. Optimizing Query Performance

- **Analyze Query Plans:** Use `EXPLAIN` or similar tools to understand query execution.
- **Avoid Full Table Scans:** Use indexes and proper filtering to avoid scanning the entire table.

Additional Resources

Online Platforms

- **HackerRank:** SQL challenges and problem-solving to practice and enhance your skills.
- **LeetCode:** SQL problems specifically designed for interview preparation.
- **SQLZoo:** Interactive SQL tutorial with exercises to practice SQL commands and queries.

Recommended Reading

- *"SQL For Dummies"* by Allen G. Taylor: A beginner-friendly guide to SQL.
- *"SQL in 10 Minutes, Sams Teach Yourself"* by Ben Forta: A quick and practical approach to learning SQL.
- *"The Practical SQL Handbook"* by Judith S. Bowman: A deeper dive into SQL best practices and advanced concepts.

SQL Tools and IDEs

- **DBeaver:** A powerful and universal database tool for developers.
- **SQLAlchemy:** A SQL toolkit and Object-Relational Mapping (ORM) library for Python.
- **MySQL Workbench:** A visual database design tool for MySQL.

Websites for SQL Best Practices and Advanced Learning

- **SQLShack:** A resource for SQL Server best practices, tips, and tutorials.
- **Mode Analytics SQL Tutorial:** A practical SQL tutorial with real-world examples.
- **Vertabelo Blog:** Articles and tutorials covering various SQL topics, from basics to advanced.

Conclusion

Mastering SQL is essential for anyone involved in data-related fields.

This CheatSheet provides an in-depth overview of SQL, from the basics to advanced topics like window functions and transaction management. By following the best practices outlined here and exploring the additional resources, you can enhance your SQL skills and become proficient in database management and querying.